It's difficult to keep data secret on a network, due to the underlying technologies. At the very bottom of the stack of hardware and software we have electronic signals that are easy to detect. A wired ethernet network has multiple participants all listening out for data: multiple machines will receive the same signal by design (and filter out their own traffic based on various address protocols). Likewise, Wi-Fi data is literally broadcast to all listeners. These days we web developers don't need to worry (too much) about encryption of the data traffic to and from our servers: protocols like HTTPS take care of the problem for us, as we'll see in Chapter 11, "Programming for the Internet." There is, however, one backend use-case for encryption that is very much our problem: password storage.

The Password Problem

In the (very, very) old days we stored and transmitted credentials (such as usernames and their corresponding passwords) in "the clear" (unencrypted plain text). With just a basic knowledge of IP networking, it was therefore trivial to extract these "secrets" when they were sent over the network or read them directly from storage (such as a filesystem or database) where possible. Today we have automatic encryption of (multiple layers of) network traffic, but we still need to store those secrets. Clearly, storing an encrypted form of a user's password is preferable: in such a system, every time the user supplies credentials, we can run the password through the same encryption process and compare the result to what we've stored. Such a process would need to be one-way; the results should not be reversible, and we've already explored hashing functions earlier in the chapter.

If we're storing hash values, then an attacker that gains access to them is still forced to perform some computation to figure out the original data. This is usually via some form of "brute-force" attack: test passwords are automatically generated, passed through the encryption algorithm, and then compared to the target hash. The following example illustrates this process by using `sha1()` to generate a target hash value for a password, then using a simple pair of `foreach` loops to build test passwords from a "dictionary" of words and computing the `sha1()` value each time. (Do not use `sha1()` for production-quality password hashing!)

```
$password = 'weaksauce';
$hash = sha1($password);
```